Part 3: PID Control - Design and Implementation

This section covers the design, tuning, and simulation of a PID controller for the Rod-Mass system.

Python Scripts Needed

- `practiceFinalSim3.py`: The main simulation script for Part 3.
- `pid_controller.py`: The implementation of the PID control law.
- `params.py`: System parameters and controller gains.

3.1 - 3.3: Controller Architecture & Gains

Description

Define the control law and choose gains based on system requirements (e.g., saturation limits and desired bandwidth).

What to do:

1. **Define Control Law**: Use the form $\tau = \tau_{ff} + K_p e - K_d \dot{y} + K_i \int edt$.
2. **Calculate τ_{ff}**: This is your equilibrium torque u_e from Part 2.1.
3. **Calculate K_p for Saturation**: Given τ_{max}, solve $\Delta\tau_{max} = K_p \Delta\theta_{max}$ to ensure the controller doesn't saturate immediately.
4. **Calculate K_d**: Choose K_d to achieve a desired damping ratio (e.g., $\zeta = 0.9$).

3.4 - 3.5: Implementation Details

Description

Implement the "dirty derivative" and anti-windup logic to handle real-world constraints.

What to do:

1. **Dirty Derivative**: In `pid_controller.py`, implement the low-pass filter for the derivative term:

$$\dot{y}_k = \frac{2\sigma - T_s}{2\sigma + T_s}\dot{y}_{k-1} + \frac{2}{2\sigma + T_s}(y_k - y_{k-1})$$

   Note: σ is typically 0.05 or $10 \times T_s$.

2. **Integrator & Anti-Windup**: Implement trapezoidal integration and back-calculation anti-windup:

$$I_k = I_k + \frac{T_s}{K_i}(u_{sat} - u_{unsat})$$

This prevents the "integral windup" effect when the actuator hits its ± 3.0 N-m limit.

3.6 - 3.8: Simulation & Tuning

Description

Run the simulation and verify performance against the nominal and uncertain models.

What to do:

1. **Run Simulation**: Execute `python practiceFinalSim3.py`.
2. **Verify Nominal Performance**: Ensure the system reaches the commanded angle (e.g., 30°) within 1-2 seconds with minimal overshoot.
3. **Verify Robustness**: The simulation automatically introduces a 10% mass uncertainty. Check if the K_i term successfully eliminates the steady-state error.
4. **Analyze Torque**: Ensure the torque τ stays mostly within the limits and doesn't chatter excessively.